

Module 01: Systems — Setting Up the Active Inference Toolkit

Course: Active Inference 101 | **Unit:** Implementation | **Audience:** First-semester undergraduates

Learning Objectives

1. Set up a **Python environment** for implementing Active Inference agents.
2. Understand the basic **data structures** needed: probability distributions, matrices, and vectors.
3. Implement a simple **generative model** from scratch using NumPy.

Introduction

Across Cognitive Science, Computational Neuroscience, and Mathematical Frameworks, we've built a conceptual and mathematical foundation for Active Inference. Now we *implement* it — turning equations into working code. This module sets up the toolkit.

Key Concepts

1. Why Python?

Python is the standard language for scientific computing:

- **NumPy** for fast matrix operations (A, B, C, D matrices)
- **SciPy** for optimization (free energy minimization)
- **Matplotlib** for visualization (plotting beliefs, free energies)
- **pymdp** — an open-source Python library specifically for Active Inference

2. Core Data Structures

Every POMDP component maps to a Python data structure:

```

import numpy as np

# A matrix:  $P(o | s)$  – observation model
# Shape: (num_observations, num_states)
A = np.array([[0.9, 0.1],      #  $P(o=0 | s)$ 
              [0.1, 0.9]])    #  $P(o=1 | s)$ 

# B matrix:  $P(s_t | s_{t-1}, a)$  – transition model
# Shape: (num_states, num_states, num_actions)
B = np.zeros((2, 2, 2))
B[:, :, 0] = np.array([[0.9, 0.1], [0.1, 0.9]]) # action 0: stay
B[:, :, 1] = np.array([[0.1, 0.9], [0.9, 0.1]]) # action 1: switch

# C vector:  $\log P(o)$  – preferences
# Shape: (num_observations,)
C = np.array([3.0, -1.0]) # prefer observation 0

# D vector:  $P(s_1)$  – initial state prior
# Shape: (num_states,)
D = np.array([0.5, 0.5]) # uniform prior

```

3. Probability Distributions in Code

Key operations on categorical distributions:

```

def normalize(x):
    """Normalize a vector to sum to 1 (make it a valid distribution)."""
    return x / x.sum()

def log_stable(x, eps=1e-16):
    """Numerically stable log – avoids log(0) = -inf."""
    return np.log(x + eps)

def softmax(x):
    """Convert log-probabilities to probabilities."""
    e_x = np.exp(x - x.max()) # subtract max for numerical stability
    return e_x / e_x.sum()

def entropy(p):
    """Compute entropy  $H(p) = -\sum p \log p$ ."""
    return -(p * log_stable(p)).sum()

def kl_divergence(q, p):
    """Compute KL divergence  $D_{KL}(q || p)$ ."""
    return (q * (log_stable(q) - log_stable(p))).sum()

```

4. Building a Generative Model Class

```
class GenerativeModel:
    """A simple Active Inference generative model."""

    def __init__(self, A, B, C, D):
        self.A = A  # observation model
        self.B = B  # transition model
        self.C = C  # preferences (log probabilities)
        self.D = D  # initial state prior

        self.num_states = A.shape[1]
        self.num_observations = A.shape[0]
        self.num_actions = B.shape[2]

    def likelihood(self, observation):
        """Return  $P(o | s)$  for a specific observation."""
        return self.A[observation, :]

    def transition(self, action):
        """Return  $P(s_t | s_{t-1})$  for a specific action."""
        return self.B[:, :, action]
```

5. Validating the Model

Important checks to ensure your model is well-formed:

```
def validate_model(A, B, C, D):
    """Check that the model matrices are valid probability distributions.
    # A columns must sum to 1
    assert np.allclose(A.sum(axis=0), 1.0), "A columns must sum to 1"
    # B columns must sum to 1 for each action
    for a in range(B.shape[2]):
        assert np.allclose(B[:, :, a].sum(axis=0), 1.0), f"B[:, {a}] columns
    # D must sum to 1
    assert np.allclose(D.sum(), 1.0), "D must sum to 1"
    print("Model valid!")
```

Summary

The Implementation track translates Active Inference mathematics into Python code. The core data structures are NumPy arrays representing the A (observation), B (transition), C (preference), and D (initial belief) components. Utility functions for normalization, softmax, entropy, and KL divergence provide the mathematical building blocks.

Further Reading

- pymdp documentation: <https://pymdp-rtd.readthedocs.io/>
- NumPy documentation: <https://numpy.org/doc/>
- Heins, C. et al. (2022). pymdp: A Python library for active inference in discrete state spaces. *JOSS*, 7(73), 4098.